Bounded Repair Controls Cache Staleness under Lost Invalidation Messages: A Controlled Study across Web Applications

Vishwak Thatikonda[1†] and Shravani Parsi[1*†]

[1*]Independent Researcher, Dublin, California, 94568, USA.

*Corresponding author(s). E-mail(s): shravaniparsi26@gmail.com;
Contributing authors: vishwakt@gmail.com;
†These authors contributed equally to this work.

Abstract

Distributed application caches for mutable web data depend on update information that may arrive late or not at all. We evaluated eight established cache policies in a controlled laboratory study spanning a mutable product service, an article feed and an administrative dashboard. The experiment used bounded synthetic request and update traces, page- or query-level responses, Redis Pub/Sub and Streams, and injected notification loss at 1%, 5% and 10%. The confirmatory campaign comprised 4,824 matched allocation blocks and 38,592 full policy executions, with 67 independent paired traces in each stratum. Compared with event-driven invalidation alone, bounded repair reduced freshness violations by 0.090 to 0.699 percentage points at 1% loss, 0.851 to 3.600 points at 5% loss and 1.478 to 7.654 points at 10% loss, while adding 6.6 to 131.2 microseconds to mean request wall time. Under reliable delivery, a write-reactive update-or-invalidate rule preserved the same freshness as fixed update and fixed invalidation, but reduced update traffic at the cost of higher request wall time relative to fixed update and showed workload-dependent CPU and origin-byte effects. These results identify bounded repair as an explicit reliability tradeoff and show that adaptive action selection moves cost between the request path, origin and notification channel without producing a universally superior policy.

Keywords: application caching, cache invalidation, distributed systems, cache consistency, reliability, performance evaluation

MSC Classification: 68M14 , 68M15 , 68M20

ORCID: Vishwak Thatikonda, 0009-0009-5828-5903; Shravani Parsi, 0009-0004-0084-4657

1 Introduction

Web caches reduce latency and origin work by reusing earlier responses, but mutable data make reuse a correctness decision as well as a performance decision. A fast response is not useful when it contains an obsolete price, an outdated article list or an inconsistent dashboard aggregate. HTTP caching distinguishes fresh and stale stored responses and defines conditional validation, while application semantics may require a stronger content-level test [1].

Applications use several established strategies to manage this boundary. Fixed time-to-live rules bound reuse by age. Stale-while-revalidate permits bounded stale serving during asynchronous refresh [2]. Conditional validation checks a revision before reuse. Event-driven policies invalidate or replace cached content after an origin update. These event policies depend on delivery: a lost notification can leave cached content stale until another mechanism repairs it.

Prior work has examined strong invalidation, adaptive TTLs, transparent database-assisted invalidation and workload-dependent selection between updating and invalidating [3–6]. None of these cited studies jointly evaluates the combination used here: matched complete application responses across controlled notification loss, multiple web workloads and measured request, origin and messaging costs. Analytical or simulation results do not by themselves establish how message construction, cache misses, origin responses and request execution interact across distinct web applications.

We address that question with a prospective controlled study of three independently implemented mutable applications. Every matched policy receives the same exogenous trace and notification-fault schedule. An application-specific oracle compares the served response with the required origin revision and content. The study reports freshness, request wall time, thread CPU, origin-response bytes and Redis payload separately. Its central finding is a two-part tradeoff: bounded repair consistently recovers freshness lost by invalidation-only policies, while write-reactive selection preserves freshness but does not dominate fixed updating or invalidation across efficiency outcomes.

2 Background and related work

2.1 HTTP and semantic freshness

RFC 9111 defines HTTP cache freshness, validation and conditions for serving stale content [1]. RFC 5861 extends those semantics with stale-while-revalidate and stale-if-error [2]. These standards govern protocol behavior, but an HTTP-fresh response can still violate an application's content rule when a cache lacks information about an origin update. We therefore define freshness from an independent semantic oracle rather than from response age or cache-hit status alone.

2.2 Invalidation, TTL selection and transparent maintenance

Cao and Liu showed that server invalidation can maintain strong consistency with competitive traffic and response cost, whereas polling adds control and latency overhead [3]. Search-cache research has used timestamps and workload signals to predict stale entries or set TTL values [4, 7]. Deep reinforcement learning has also been applied to TTL-based cache-utility control under changing content popularity [8]. Ji et al. instead use database and cache changes to support precise transparent invalidation for application-level query caches [5]. These results define fixed, adaptive and transparent baselines for the controlled comparison.

2.3 Write-reactive update or invalidation

Mao et al. formulate the choice between updating and invalidating as a function of expected writes between reads and the respective update, invalidation and miss costs [6]. Their analysis shows that neither fixed action is uniformly preferable. Following their cost comparison, we implement the cost-minimizing rule: update when the expected update cost is lower than the combined cost of invalidation and the subsequent miss; otherwise, invalidate. This comparison is especially relevant for query caches because a complete pushed replacement can be much larger than an invalidation message.

2.4 Notification delivery

Redis Pub/Sub provides at-most-once delivery; a message that cannot be received is lost [9]. Redis Streams persists entries and supports acknowledgment and recovery [10]. Our study uses both transports, with one controlled consumer and a study-owned fault adapter. Injected loss is an experimental treatment, not a claim about the transports' native failure rates.

3 Research questions

The study posed three questions:

1. Under reliable notification delivery, how do established cache policies differ in freshness, request time, origin work and control traffic?
2. How does notification loss change event-driven invalidation, and how much freshness does bounded repair recover?
3. Which tradeoffs reproduce across distinct applications, request/update regimes and Redis delivery modes?

The confirmatory analysis comprises five prespecified contrast types. Three compare bounded repair with always invalidation at 1%, 5% and 10% notification loss. Two compare the write-reactive rule with always update and always invalidation under no injected fault. The remaining policies and fault mechanisms provide training, validation and descriptive context.

4 Workloads and policies

4.1 Mutable applications and semantic oracles

The study uses three deterministic mutable applications. The product workload returns individual products with revisioned price, availability and descriptive fields. The article-feed workload adapts the pinned SvelteKit RealWorld application to a controlled origin whose responses include membership, order, tags and displayed article fields [11]. The dashboard workload adapts the pinned React-admin e-commerce demo to a controlled REST origin and evaluates orders, revenue, pending reviews and customer references at one revision [12].

Each workload implements reset, commit, request and oracle operations. The oracle compares the complete declared response content with the state required at the request decision. Version labels alone do not determine freshness. Feed membership, ordering and displayed fields matter for the article workload; joined counts and referenced entities matter for the dashboard.

4.2 Cache policies

Every allocation executes eight policies:

1. **Bypass** reads the origin for every request.
2. **Conditional validation** checks a revision validator before reuse.
3. **Fixed TTL** reuses content until a training-selected age expires.
4. **Stale-while-revalidate** permits reuse within a selected stale window while refreshing asynchronously.
5. **Always invalidation** removes affected cached responses after a delivered update notification.
6. **Always update** pushes complete replacement content for affected cached queries.
7. **Write-reactive selection** chooses update or invalidation from observed writes between reads and training-derived serialized-byte costs.
8. **Invalidation with bounded repair** combines delivered-event invalidation with a maximum cached age selected during training.

For the article and dashboard workloads, update messages carry complete replacements for affected cached queries at the commit revision. Message construction reads and bytes are charged to the update policy. Delayed delivery cannot consult a newer origin state.

5 Experimental design and analysis

5.1 Traces, regimes and notification faults

Every trace contains 100 origin updates. Read-heavy, balanced and write-intensive traces contain 500, 200 and 100 requests, respectively. Inter-event gaps range from 5 to 15 logical milliseconds. The traces are bounded synthetic workloads designed to preserve complete request, update, revision and content identities; they do not estimate production prevalence.

The confirmatory campaign includes no injected fault and exact notification-loss rates of 1%, 5% and 10%. The corresponding schedules drop 0, 1, 5 or 10 of the 100 update notifications. Redis Pub/Sub and Streams both carry source-bound notification envelopes through an isolated local server. Matched policies receive identical exogenous request, update and loss schedules.

5.2 Evidence partitions and selected parameters

Engineering, training, held-out validation, variance planning and final testing use disjoint seed ranges. Training selected TTL, stale-window, repair-age and initial expected-writes values. Held-out validation tested these choices without refitting. Variance planning used ten paired traces per stratum and the prospective precision rule

$$n = \left\lceil \left(\frac{2.5758293 s}{h} \right)^2 \right\rceil,$$

where s is the paired planning standard deviation and h is the target interval half-width. The maximum of 450 calculations required 67 final-test seeds. No final-test observation contributed to parameter selection or sample-size determination.

5.3 Allocation and experimental unit

One allocation block is defined by workload, regime, trace seed, transport and fault profile. It executes all eight policies in a deterministic hash-derived order independent of trace contents. Each policy runs in a new process after origin, cache, notification and policy state are reset. The runtime is Ubuntu 24.04 with Python 3.9.25 and Redis 7.0.15. At most three blocks execute concurrently.

The independent experimental unit is one complete policy execution over one trace. Requests within a trace are repeated observations, not independent replicates. The final campaign contains 4,824 blocks and 38,592 policy executions.

5.4 Outcomes

The admissibility outcome is the proportion of requests that violate the workload-specific semantic freshness rule. The primary efficiency outcomes are mean request wall time, total operation-thread CPU per trace, request-path plus publisher origin-response bytes and delivered Redis payload bytes. These quantities remain separate; no weighted score combines freshness, time, CPU and bytes.

5.5 Statistical analysis

Primary contrasts are paired by workload, regime, transport, fault profile and trace seed. Each stratum contains 67 complete paired trace units. We report the paired mean difference and a deterministic 99% percentile bootstrap interval from 10,000 resamples of complete trace pairs. Efficiency is interpreted only when the upper 99% bound for excess freshness violations is no greater than 0.01. No P values, significance stars or

binary significance claims are used. The 99% intervals describe individual estimands and are not simultaneous familywise intervals across all 450 estimates.

5.6 AI assistance

OpenAI ChatGPT through Codex assisted with code development and debugging.

6 Results

The experiment completed all 4,824 planned allocation blocks and all 38,592 policy executions. Every primary estimate contains 67 complete paired traces. All 90 freshness comparisons met the prespecified admissibility rule, so the associated efficiency outcomes were interpretable.

6.1 Bounded repair reduces freshness violations under notification loss

Bounded repair reduced the freshness violations caused by lost invalidation messages in every evaluated stratum (Fig. 1). At 1% notification loss, the paired mean reduction ranged from 0.090 to 0.699 percentage points across workloads, regimes and transports. The 99% interval was below zero in 17 of 18 strata and included zero in one. At 5% loss, the reduction increased to 0.851 to 3.600 points; at 10% loss it increased to 1.478 to 7.654 points. All intervals at 5% and 10% loss were below zero.

The repair mechanism imposed a consistent request-path cost. Mean request wall time increased by 6.6 to 120.6 microseconds at 1% loss, 6.8 to 124.5 microseconds at 5% loss and 7.6 to 131.2 microseconds at 10% loss. All 54 intervals were above zero. Thread CPU increased by 0.47 to 65.28 milliseconds per trace, with 53 intervals above zero and one including zero. Origin-response bytes also increased as repair recovered from missing messages. Delivered Redis payload was identical to always invalidation because both policies used the same invalidation envelopes. Bounded repair therefore exchanges periodic validation or refresh work for a progressively larger freshness benefit as loss rises.

6.2 Write-reactive selection shifts cost rather than dominating fixed actions

Under reliable delivery, write-reactive selection, always update and always invalidation produced identical freshness-violation proportions in all 36 primary comparisons. The efficiency results therefore reflect different ways of maintaining the same observed freshness in these conditions (Fig. 2).

Compared with always update, write-reactive selection increased mean request wall time in all 18 strata by 5.5 to 306.5 microseconds. Every 99% interval was above zero. It reduced delivered Redis payload in every stratum, by 19.2 to 1,759.1 KiB per trace, because it avoided some complete replacement messages. CPU and origin bytes divided by workload. Both quantities were lower for all 12 article-feed and dashboard strata and higher for all six product strata, with every interval excluding zero. Complete

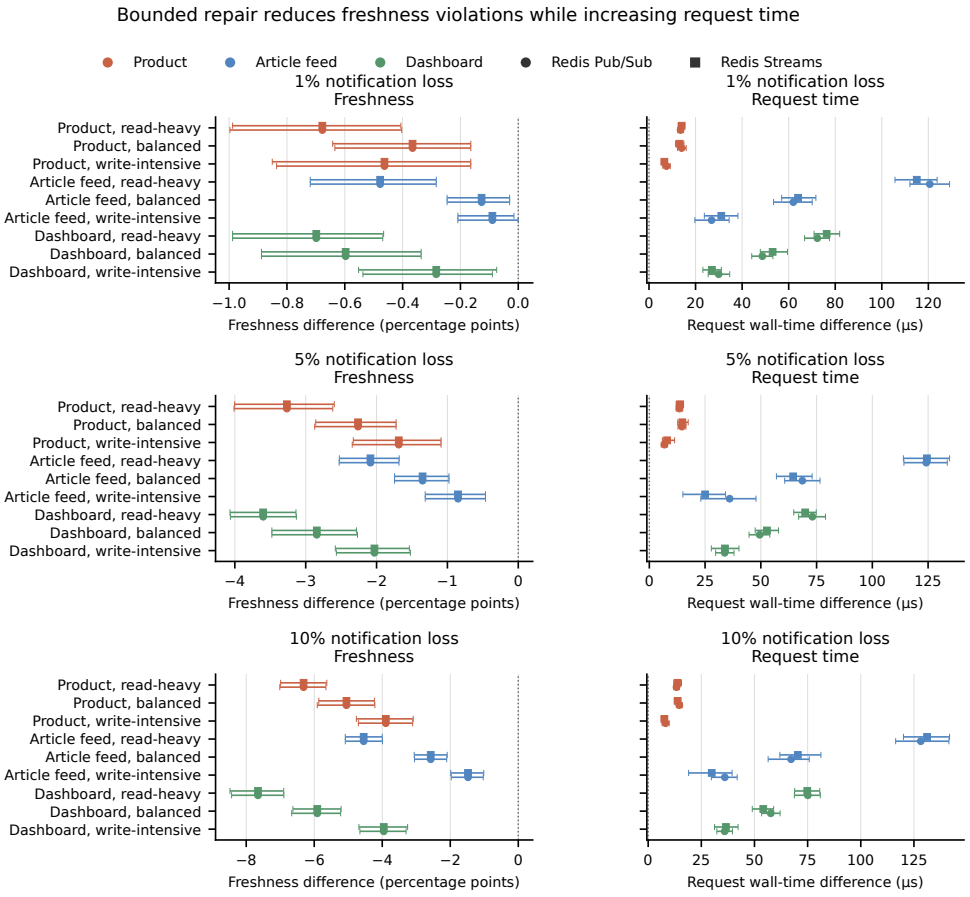


Fig. 1 Bounded repair reduces freshness violations while increasing request time. Points are paired mean differences between invalidation with bounded repair and always invalidation. Horizontal bars are 99% percentile bootstrap intervals from 10,000 resamples of 67 complete paired traces. The left column reports the freshness-violation difference at 1%, 5% and 10% notification loss. The right column reports the corresponding mean request wall-time difference. Negative freshness values favor repair; positive request-time values indicate additional cost. Colors identify workloads and marker shapes identify Redis transport. Each marker represents 67 complete paired traces

query replacements were costly for the article and dashboard, while the smaller product updates made fixed updating comparatively cheap.

The comparison with always invalidation did not yield a uniform efficiency gain. Write-reactive selection reduced origin-response bytes only in the six product strata, by 0.84 to 1.90 KiB per trace, and matched fixed invalidation in the other 12 strata. It added 5.9 to 11.9 KiB of Redis payload in every stratum. Request-time intervals were above zero in seven strata, below zero in one and included zero in ten. CPU intervals were above zero in seven, below zero in five and included zero in six. The adaptive rule therefore selected a workload-dependent position between the fixed actions, but no evaluated efficiency outcome established universal dominance.

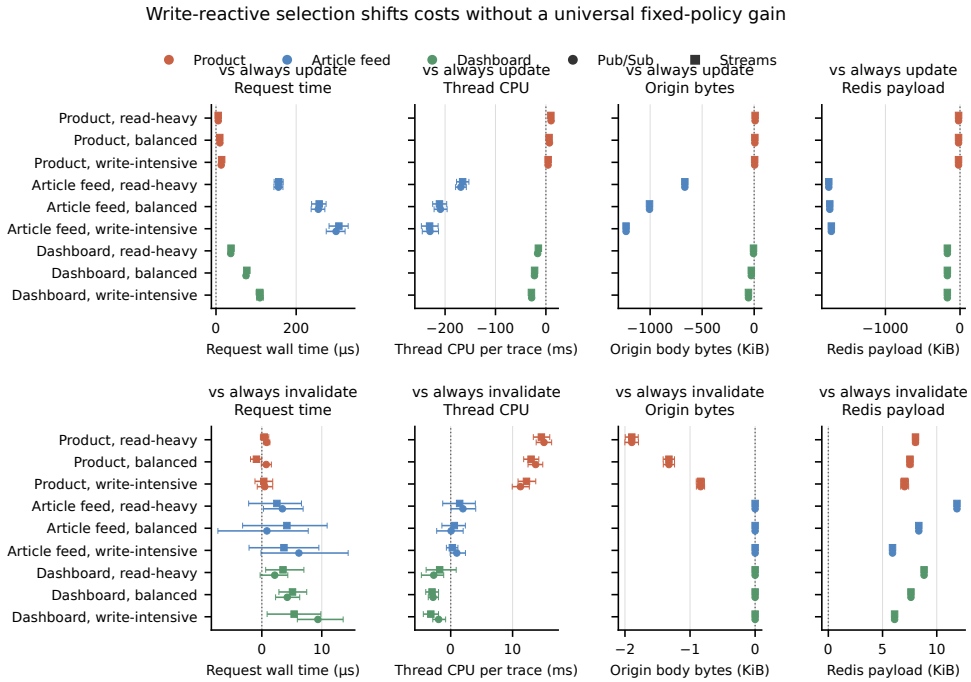


Fig. 2 Write-reactive selection shifts costs without a universal fixed-policy gain. Points are paired mean differences for write-reactive selection minus always update in the top row and minus always invalidation in the bottom row. Columns report request wall time, operation-thread CPU per trace, total origin-response bytes and delivered Redis payload. Horizontal bars are 99% percentile bootstrap intervals from 10,000 resamples of 67 complete paired traces. Negative values indicate lower cost for write-reactive selection. Colors identify workloads and marker shapes identify Redis transport. Freshness-violation differences were zero in all displayed strata

7 Discussion

This study separates two questions that are often combined under cache adaptation. The first is reliability after an update signal fails. Bounded repair directly addresses that failure and recovered progressively more freshness as notification loss increased. The second is whether an application should push replacement content or invalidate and pay a later miss. The write-reactive rule changed that allocation of work but did not supply a uniformly better answer across request time, CPU, origin bytes and notification traffic.

The repair result gives application designers a concrete choice. Event invalidation without repair is inexpensive when delivery is reliable, but a lost message can extend stale residence until another event changes the entry. A bounded maximum age converts that open-ended risk into periodic work. The data show both sides of this choice: freshness gains reproduced across all three applications and both Redis modes, while request time and CPU increased in nearly every stratum. Repair is therefore a reliability mechanism with a measured operating cost, not a cost-free enhancement to invalidation.

The write-reactive result extends the analytical update-or-invalidate tradeoff into complete application responses [6]. For large query-complete replacements, avoiding updates substantially reduced publisher and Redis bytes and lowered CPU, but subsequent misses moved work onto the request path. For small product objects, fixed updating avoided those misses at comparatively low message cost. This explains why a rule that minimizes a declared byte-cost expression can reduce some resource measures while increasing request wall time. A deployment must choose which resource is constrained rather than treating adaptation itself as the objective.

The conclusions are bounded by the experimental design. The three workloads cover object, feed and joined-dashboard responses, but they do not represent the frequency of production cache patterns. The traces are synthetic and bounded, so the estimates describe the declared request/update regimes rather than population prevalence. GitHub-hosted runners and local Redis instances control software and allocation identity but do not reproduce wide-area networks or multi-region failover. The treatments inject exact delivery faults at the study adapter; they do not estimate native Pub/Sub or Streams failure rates. Prospective partitions, complete-content oracles, matched traces, two Redis modes and independently implemented workloads reduce several internal-validity risks, but deployment-specific rates and costs still require local measurement.

Within those boundaries, the evidence changes the practical order of evaluation. A cache policy should first demonstrate semantic freshness under the delivery failures it can experience. Efficiency comparisons are meaningful only among policies that satisfy the freshness criterion. Once freshness is controlled, update, invalidation and repair distribute cost differently across messages, origin work and the request path. Reporting those quantities separately is more informative than collapsing them into one reward or utility score.

8 Conclusion

Lost update notifications turn event-driven cache invalidation into a freshness risk. Across three controlled web applications, bounded repair reduced that risk at a consistent request-time and CPU cost. Write-reactive selection preserved freshness under reliable delivery and reduced large replacement messages, but its request, CPU and origin effects depended on workload and comparator. These results support a practical sequence: first establish semantic freshness under expected delivery faults, then decide where to pay the remaining costs across messaging, origin work and request processing.

Statements and Declarations

Funding

The authors did not receive support from any organization for the submitted work.

Competing interests

The authors have no relevant financial or non-financial interests to disclose.

Ethics approval and consent to participate

Not applicable. This study did not involve human participants, human data or animals.

Consent for publication

Not applicable.

Data availability

Analysis-ready paired estimates, figure-source data, confirmatory analysis outputs and the archive inventory are available in Zenodo [13]. All 4,824 raw execution archives are available in the associated GitHub release, together with checksums and the exact source revision.

Code availability

Study code, prespecified protocols, selected parameter files and the analysis implementation are available in the project repository at tag `freshness-caching-study-v1.0.0`.

Author contributions

Vishwak Thatikonda and Shravani Parsi contributed equally to the study and to preparation of the manuscript. Both authors read and approved the final manuscript.

References

[1] Fielding, R.T., Nottingham, M., Reschke, J.: Http caching. RFC 9111, RFC Editor (2022). https://doi.org/10.17487/RFC9111

[2] Nottingham, M.: Http cache-control extensions for stale content. RFC 5861, RFC Editor (2010). https://doi.org/10.17487/RFC5861

[3] Cao, P., Liu, C.: Maintaining strong cache consistency in the world wide web. IEEE Transactions on Computers **47**(4), 445–457 (1998) https://doi.org/10.1109/12.675713

[4] Sazoglu, F.B., Cambazoglu, B.B., Ozcan, R., Altingovde, I.S., Ulusoy, O.: Strategies for setting time-to-live values in result caches. In: Proceedings of the 22nd ACM International Conference on Information and Knowledge Management, pp. 1881–1884 (2013). https://doi.org/10.1145/2505515.2507886

[5] Ji, Y., Zhou, X., Zhou, Y., Wang, K.: The Case of Transparent Cache Invalidation in Web Applications (2023). https://arxiv.org/abs/2311.02384

[6] Mao, Z., Iyer, R., Shenker, S., Stoica, I.: Revisiting cache freshness for emerging real-time applications. In: Proceedings of the 23rd ACM Workshop on Hot Topics in Networks (2024). https://doi.org/10.1145/3696348.3696858

[7] Alici, S., Altingovde, I.S., Ozcan, R., Cambazoglu, B.B., Ulusoy, O.: Timestamp-based result cache invalidation for web search engines. In: Proceedings of the 34th International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 973–982 (2011). https://doi.org/10.1145/2009916.2010046

[8] Cho, C., Shin, S., Jeon, H., Yoon, S.: Ttl-based cache utility maximization using deep reinforcement learning. In: 2021 IEEE Global Communications Conference, pp. 1–6 (2021). https://doi.org/10.1109/GLOBECOM46510.2021.9685845

[9] Redis Ltd.: Redis Pub/Sub. Redis documentation. Accessed 2026-10-29. https://redis.io/docs/latest/develop/pubsub/

[10] Redis Ltd.: Redis Streams. Redis documentation. Accessed 2026-10-29. https://redis.io/docs/latest/develop/data-types/streams/

[11] Svelte contributors: SvelteKit RealWorld Application. GitHub repository, commit df796708040f5200ec572b28ab7f88ecee5794dd (2026). https://github.com/sveltejs/realworld/tree/df796708040f5200ec572b28ab7f88ecee5794dd

[12] Marmelab: React-admin E-commerce Demo. GitHub repository, commit 0c33d9c3c27cc4bf97e11f56080c5bfe182f2f2a (2026). https://github.com/marmelab/react-admin/tree/0c33d9c3c27cc4bf97e11f56080c5bfe182f2f2a/examples/demo

[13] Thatikonda, V., Parsi, S.: Freshness-Aware Caching Controlled Study: Analysis and Reproducibility Package. Zenodo (2026). https://doi.org/10.5281/zenodo.23843162